Exp No: 29

PRIORITY-BASED INTERRUPT HANDLING

AIM:

To model and simulate priority-based interrupt handling in SystemC by servicing high-priority and low-priority interrupts.

PROCEDURE:

1. Create high-priority and low-priority interrupt events using sc_event.
2. Define separate interrupt service routines (ISRs) using SC_THREAD.
3. Create a generator process to trigger the low-priority interrupt first, followed by the high-priority interrupt.
4. Use wait() to simulate the processing time of each interrupt service routine.
5. Run the simulation for 10 seconds and observe the interrupt servicing order and execution times.

PROGRAM:

```
#include <systemc.h>


SC_MODULE(priority_interrupt) {


sc_event low_int, high_int;


void low_ISR() {

while(true) {

wait(low_int);

cout << "Low Priority Interrupt at "

<< sc_time_stamp() << endl;

wait(3, SC_SEC);

}

}


void high_ISR() {
```

```
while(true) {
wait(high_int);
cout << "High Priority Interrupt at "

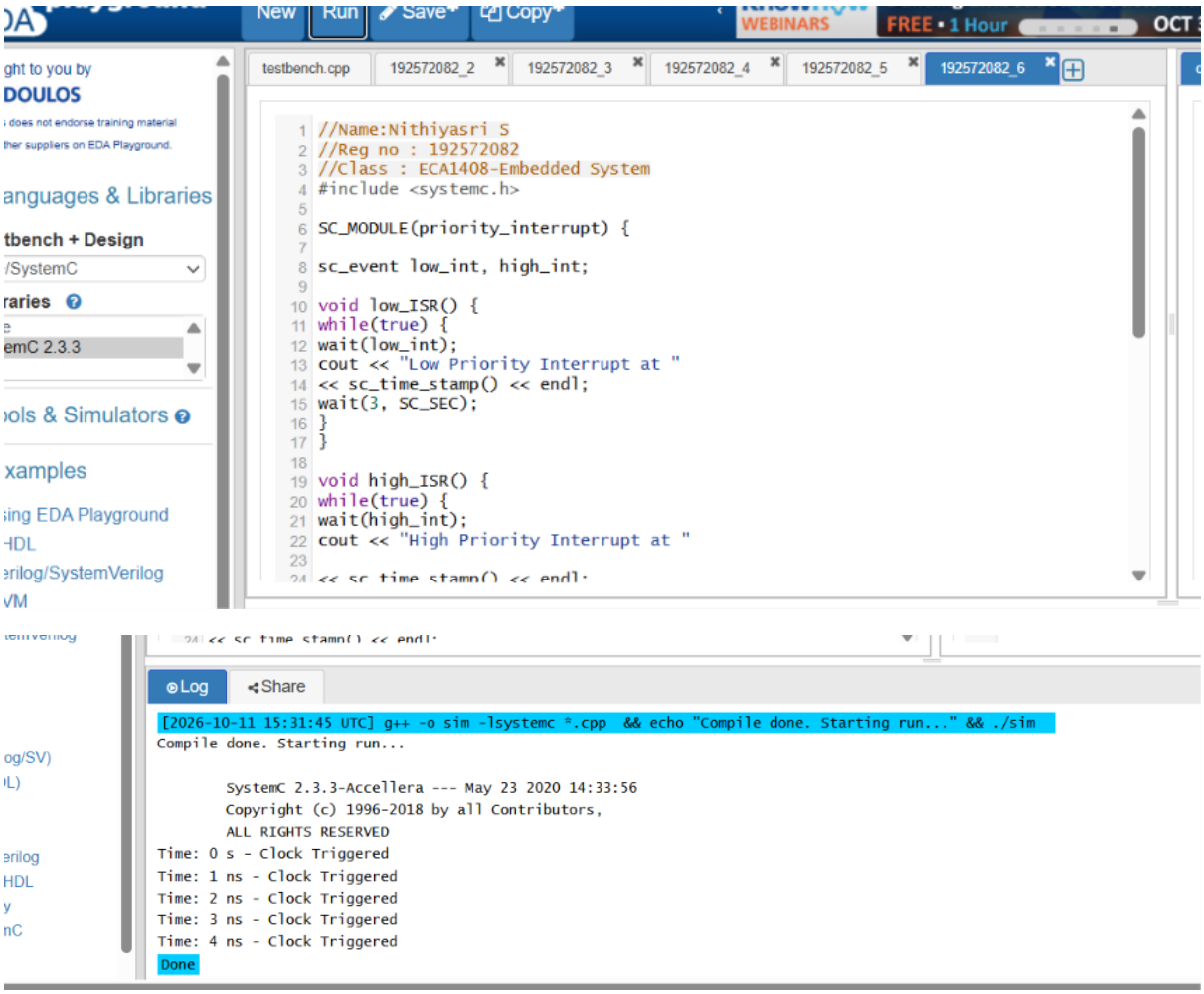
<< sc_time_stamp() << endl;
wait(1, SC_SEC);
}
}

void generator() {
wait(1, SC_SEC);
low_int.notify();

wait(1, SC_SEC);
high_int.notify();
}

SC_CTOR(priority_interrupt) {
SC_THREAD(low_ISR);
SC_THREAD(high_ISR);
SC_THREAD(generator);
}
};

int sc_main(int argc, char* argv[]) {
priority_interrupt obj("Priority_INT");
sc_start(10, SC_SEC);
return 0;
}
```



RESULT:

Thus, priority-based interrupt handling was modeled and simulated using SystemC, and the execution of high-priority and low-priority interrupt service routines was verified.